

LABORATORY OBSERVATION

Course: Full Stack Web Development Course

Code: 23CM4121

Experiment No.: Observation Task-06

Student Name: Ganji Venkata Vijay Anudeep

Roll No.: A24126552081

Date: 25-09-2026

1. TITLE

Comprehensive Project-Based Learning: JavaScript OOP, Node.js Modules, Package Management, and Express.js Middleware

2. AIM

To develop, execute, and evaluate foundational back-end and JavaScript programming concepts, specifically focusing on object-oriented structures, built-in Node.js module implementation, external package management using NPM, and the request-response cycle utilizing Express.js middleware.

3. OBJECTIVE

- To differentiate between traditional JavaScript constructor functions and modern ES6 classes for object creation.
- To implement and understand the purpose of Node.js built-in modules (`os`, `path`, and `fs`).
- To initialize a Node.js project, manage dependencies via `package.json`, and utilize external packages.
- To configure an Express.js server and implement custom middleware for tracking incoming HTTP requests.
-

4. THEORY

- **JavaScript OOP:** A traditional constructor function acts as a reusable blueprint, attaching methods via the prototype chain. ES6 Classes provide syntactic sugar over this prototype-based inheritance, offering a cleaner, more intuitive structure using `constructor()` to initialize shared properties and defining methods directly within the class body.
- **Node.js Modules:** Node.js operates on a modular architecture. Built-in modules like `os` allow access to operating system information, `path` ensures cross-platform compatibility when resolving file directories, and `fs` (File System) enables synchronous and asynchronous reading, writing, and deleting of files.
- **NPM & package.json:** NPM (Node Package Manager) handles external dependencies. The `package.json` file serves as a manifest for project metadata. External modules (like `uuid` for unique identifiers) must be installed via the terminal and imported into the application using `require()`.

- **Express.js Middleware:** In Express.js, middleware functions intercept the request-response cycle. They have access to the request (`req`) and response (`res`) objects. Middleware can execute logic (like logging timestamps or routes) and must invoke the `next()` function to pass control to the subsequent handler to prevent the request from hanging.

5. ALGORITHM/PROCEDURE

1. Read the functional requirements for each task thoroughly.
2. **For Task 5 (JS Classes):** Define a traditional constructor function and an ES6 class. Instantiate unique objects using the `new` keyword and invoke their methods to demonstrate shared logic.
3. **For Task 6 (Node Modules):** Import the `os`, `path`, and `fs` modules. Retrieve system memory/platform details, construct a file path safely, write a string to a text file, read the file back into memory, and log the outputs.
4. **For Task 7 (NPM):** Initialize a project environment using `npm init -y`. Install the `uuid` package via `npm install uuid`. Require the package in the script and execute its core function to generate a secure session token.
5. **For Task 8 (Express Middleware):** Install Express.js. Initialize an Express application and define a custom middleware function that logs the current timestamp, request method, and URL. Apply the middleware globally using `app.use()`, define a root GET route, and start the server on port 3000.
6. Execute each script in the terminal and verify that the output matches the expected behavior.

Task 5: JavaScript Functions vs. Classes

6. PROGRAM

// TASK 6: BUILT-IN NODE.JS MODULES

```
const os = require('os');
const path = require('path');
const fs = require('fs');
```

// 1. OS Module Example

```
console.log(`OS Platform: ${os.platform()}`);
console.log(`Total Memory: ${os.totalmem()} bytes`);
```

// 2. Path Module Example

```
// Safely joins directories regardless of the operating system
const constructedPath = path.join(__dirname, 'logs', 'app.log');
console.log(`Constructed Path: ${constructedPath}`);
```

// 3. FS (File System) Module Example

```
const filePath = 'test-file.txt';
```

```
// Write to a file
fs.writeFileSync(filePath, 'Hello, Node.js File System!');
// Read from the file
const fileContent = fs.readFileSync(filePath, 'utf8');

console.log(`File Content: ${fileContent}`);

// Cleanup for testing purposes
fs.unlinkSync(filePath);
```

7. CODE EXPLANATION

Code	Explanation
<code>function UserFunction()</code>	Traditional way to create an object constructor before ES6.
<code>.prototype</code>	Used to attach shared methods to a constructor function to save memory.
<code>class UserClass</code>	Modern ES6 syntax defining a blueprint for creating objects.
<code>constructor(name, role)</code>	A special method in a class automatically called upon object instantiation to assign properties.
<code>new</code>	Keyword used to create a new instance of an object from a constructor or class.

8. TEST CASES AND EXECUTION DATA/OUTPUT

Test Case	Input/Action	Expected Result	Actual Result	Status

TC-01	Instantiate <code>user1</code> with ("Alice", "Admin")	Object created with corresponding properties	Object created successfully	Pass
TC-02	Call <code>user1.displayProfile()</code>	Console logs: "User: Alice Role: Admin"	Output matches expected string	Pass
TC-03	Call <code>user2.displayProfile()</code>	Console logs: "User: Bob Role: Editor"	Output matches expected string	Pass

Task 6: Node.js Built-in Modules

6. PROGRAM

JavaScript

```
// TASK 6: BUILT-IN NODE.JS MODULES
```

```
const    require 'os'
```

```
const    require 'path'
```

```
const    require 'fs'
```

```
// 1. OS Module Example
```

```
console   `OS Platform: ${os.platform()}`
```

```
console   `Total Memory: ${os.totalmem()} bytes`
```

```
// 2. Path Module Example
```

```
// Safely joins directories regardless of the operating system
```

```
const                                           'logs' 'app.log'
```

```
console   `Constructed Path: ${constructedPath}`
```

```
// 3. FS (File System) Module Example
```

```
const     'test-file.txt'
```

```
// Write to a file
```

```
           'Hello, Node.js File System!'
```

```
// Read from the file
```

```
const                                           'utf8'
```

```
console   `File Content: ${fileContent}`
```

```
// Cleanup for testing purposes
```

7. CODE EXPLANATION

Code	Explanation
<code>require('os')</code>	Imports the built-in operating system module.
<code>os.totalmem()</code>	Returns the total amount of system memory in bytes.
<code>path.join()</code>	Concatenates path segments safely, handling OS-specific delimiters (/ vs \).
<code>fs.writeFileSync()</code>	Synchronously writes data to a specified file, creating it if it doesn't exist.
<code>fs.readFileSync()</code>	Synchronously reads the contents of a file. Uses 'utf8' to return a string.

8. TEST CASES AND EXECUTION DATA/OUTPUT

Test Case	Input/Action	Expected Result	Actual Result	Status
TC-01	Execute script to log OS info	Prints OS platform (e.g., linux, win32) and memory	Logs correct OS details	Pass
TC-02	Execute <code>path.join</code>	Constructs an absolute path ending in <code>logs/app.log</code>	Path constructed cleanly	Pass

TC-03	Write to and read from <code>test-file.txt</code>	File is created, reads "Hello, Node.js File System!"	Content written and read accurately	Pass
-------	---	--	-------------------------------------	------

Task 7: NPM and package.json

6. PROGRAM

JavaScript

// TASK 7: EXTERNAL PACKAGES (Ensure 'npm install uuid' is run first)

// Importing the installed external package

```
const v4          require 'uuid'
```

```
function createNewSession
```

// Utilizing the external package to generate a secure UUID

```
const
```

```
console.log(`New Session Generated: ${sessionToken}`)
```

// Execution / Output Demonstration

7. CODE EXPLANATION

Code	Explanation
<code>npm init -y</code>	(Terminal Command) Generates a default <code>package.json</code> file.
<code>npm install uuid</code>	(Terminal Command) Downloads the external <code>uuid</code> package into <code>node_modules</code> .
<code>require('uuid')</code>	Imports the external module into the current JavaScript file.
<code>uuidv4()</code>	Function from the package that generates a random, cryptographically secure version 4 UUID.

8. TEST CASES AND EXECUTION DATA/OUTPUT

Test Case	Input/Action	Expected Result	Actual Result	Status
TC-01	Verify <code>package.json</code>	Contains <code>"uuid"</code> under <code>"dependencies"</code>	Dependency listed correctly	Pass
TC-02	Execute <code>createNewSession()</code>	Console logs a standard 36-character UUID string	Output: e.g., 1b9d6bcd-bbfd-4b2d-9b5d...	Pass
TC-03	Call <code>createNewSession()</code> twice	Generates two distinct UUIDs	Tokens are unique	Pass

Task 8: Express.js Middleware

6. PROGRAM

JavaScript

```
// TASK 8: EXPRESS.JS MIDDLEWARE
// Note: Ensure Express is installed via 'npm install express'
const      require 'express'
const
const      3000

// Middleware for logging incoming requests
const
  // Capture the current time, HTTP method, and requested URL
  const      new Date
  console    `[${timestamp}] ${req.method} request made to: ${req.url}`

  // CRITICAL: Pass control to the next middleware or route handler

// Mount the middleware globally so it runs on every incoming request

// Define a basic route
  '/'
    200      'Middleware testing successful!'
```

```
// Start the server
```

```
console    `Server is actively running on http://localhost:${PORT}`
```

7. CODE EXPLANATION

Code	Explanation
<code>express()</code>	Initializes a new Express application instance.
<code>app.use()</code>	Mounts a middleware function globally, executing it for every incoming request.
<code>req.method</code> & <code>req.url</code>	Extracts the HTTP method (GET, POST) and the specific endpoint requested.
<code>next()</code>	Callback function that hands off control to the next middleware or route handler in the stack.
<code>app.listen()</code>	Binds and listens for connections on the specified network port.

8. TEST CASES AND EXECUTION DATA/OUTPUT

Test Case	Input/Action	Expected Result	Actual Result	Status
TC-01	Run script	Terminal displays "Server is actively running on http://localhost:3000"	Server boots successfully	Pass

TC-02	Access http://localhost:3000/ in browser	Browser displays "Middleware testing successful!"	Response sent successfully	Pass
TC-03	Check Terminal after browser request	Console logs timestamp, "GET", and "/" path	Log format generated accurately	Pass

9. RESULT

Thus, the technical implementations for Project-Based Learning assignments covering ES6 Object-Oriented principles, core Node.js modular functionality, NPM package management workflows, and Express.js middleware server configuration were successfully developed, tested, and validated against their expected outputs.